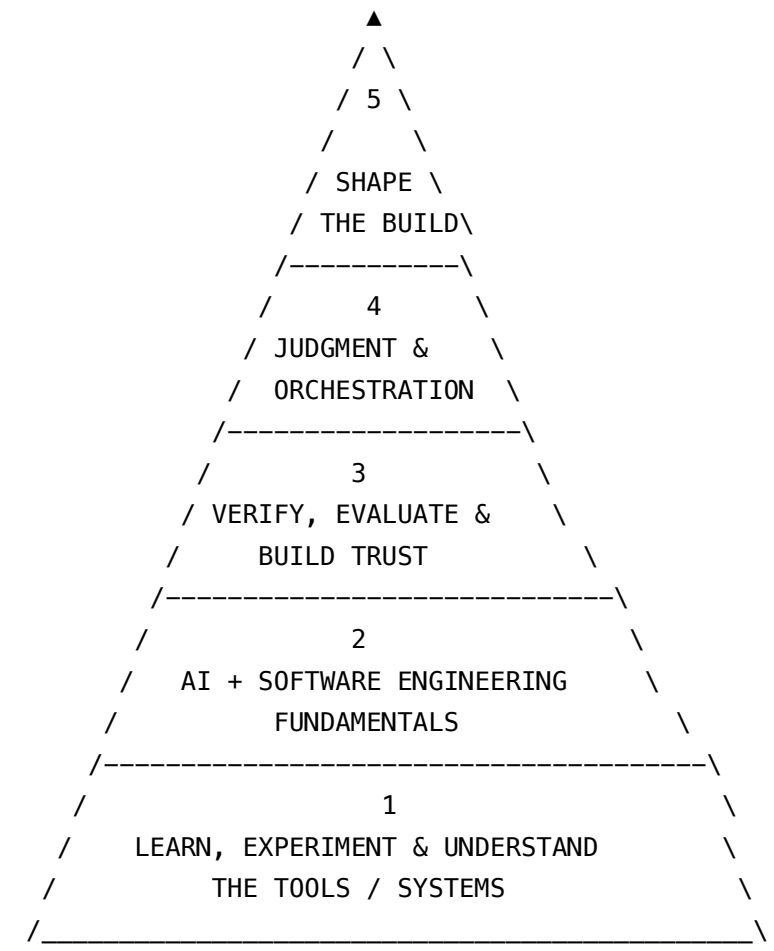
The AI Engineer's Pyramid of Needs

A strong reaction is that Andrew Ng's coding-agent map is really describing a **shift in the scarcity structure of engineering**.

Coding is becoming abundant. **Judgment, verification, context, and choosing what to build are becoming scarce.** Ng's detailed coding-agent workflow—planning, directing the workflow, enabling autonomy, reviewing work, and customizing the agent/environment—supports exactly that interpretation. He also explicitly warns against confusing long autonomous runs with effective engineering: skilled human intervention and iterative steering still produce better results. (Thread Navigator)

That suggests a nice adaptation of Maslow:

The AI Engineer's Maslow Pyramid



I would give the five layers slightly more provocative names:

1 — Grounding: "Know what you are standing on." Software engineering fundamentals, AI fundamentals, how agents actually work, context windows, tools, retrieval, har-

nesses, permissions and failure modes. This is the physiological layer: without it, everything above is fragile. Ng explicitly argues that a mental model of the agent makes it less of a black box and lets engineers diagnose behaviours such as overengineering, weak verification or destructive actions. (Thread Navigator)

2 — Capability: "Be able to build without the magic." Architecture, data, APIs, security, reliability, deployment—and building actual AI applications. This is important because agents don't make fundamentals obsolete; they make ignorance of fundamentals **scale faster**. The wider skills map specifically retains software engineering fundamentals alongside AI application development rather than replacing them with prompting. (HyperIndex 日报)

3 — Trust: "Don't ask whether it wrote the code. Ask whether you know it works." Tests, evals, verifiers, observability, review, acceptance criteria and safe operating boundaries. This may be the most underrated layer. Once generating a solution costs seconds, **verification becomes more valuable than generation**. A good AI engineer increasingly designs the feedback environment in which an agent can discover that it is wrong and correct itself.

4 — Leverage: "Manage intelligence rather than keystrokes." Planning, decomposition, context engineering, autonomy, parallel agents, knowing when to intervene, preserving state and customizing the environment. This is where the article gets particularly interesting: expert agent use isn't "prompting better." It is much closer to **management and systems design for artificial workers**. Ng describes effective use as deciding the human/agent division of labour, decomposing work into verifiable steps, coordinating parallel agents and accumulating learnings across sessions. (Thread Navigator)

5 — Agency / Self-actualization: "Build the right thing." Problem selection, product sense, customer understanding, business context, architecture-level judgment and ownership of outcomes: **shaping the build**. In the original broader skills map, Ng makes this a top-level AI-engineering competency because increasingly capable agents can execute a clear spec—so the engineer's comparative advantage moves toward deciding what belongs in the spec. (HyperIndex 日报)

The insight I'd put next to the pyramid

> **As AI lowers the cost of implementation, the AI engineer moves up the abstraction stack: from producing code → proving correctness → directing agents → exercising judgment → choosing worthwhile problems.**

Or even shorter:

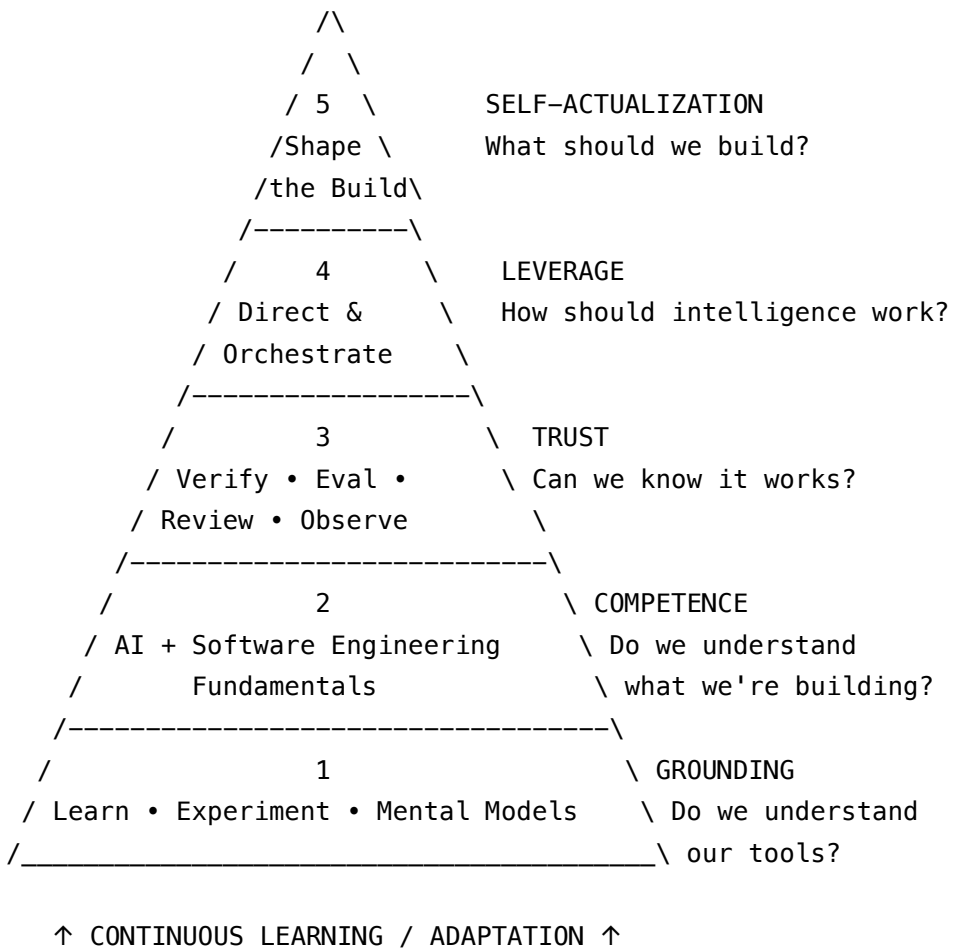
> **The AI Engineer's career ladder is shifting from "Can you build it?" to "Can you make AI build it reliably?" to "Do you know what should be built?"**

I think that is more interesting than simply mapping Ng's categories one-for-one onto Maslow.

There is also a useful **Maslow-esque dependency** here. Someone can jump straight to autonomous coding agents and appear spectacularly productive. But if the layers underneath are weak, the pyramid is inverted: they are generating more code, more quickly, that they are less capable of evaluating. In that sense, **AI amplifies the level beneath it**. Strong fundamentals become leverage; weak fundamentals become accelerated technical debt. This aligns with the broader observation around Ng's map that when agents can produce large amounts of plausible code quickly, judgment about what should exist and whether it is correct becomes more—not less—important. (SmartScale)

And I would draw **"continuous learning" not as another layer but as the soil beneath / an arrow running up the side of the whole pyramid**. Ng's broader map treats continuous learning as underlying all four major skill areas, and his coding-agents piece makes the reason especially clear: agent capabilities and best practices are changing too quickly for mastery of a particular tool to remain durable. (Thread Navigator)

So visually I'd probably make the final version something like:



The slightly cheeky title could be **“Maslow’s Hierarchy of AI Engineering Needs”**, with the caption:

“Vibe coding is near the bottom, not the top.”

That last line captures what I think is the genuinely insightful reaction to Ng’s piece: the visible novelty is the agent writing the code, but **the differentiating skills increasingly sit above the act of coding itself.**

What I think

Great, I have these thoughts:

- I would rename it to just “AI Engineer Pyramid” and drop the Maslov reference.
- change the Capability sub header to “Be able to build with and without the magic” (I think being able to build with AI is an important capability on it’s own
- the most important insight I had:
  - The grounding is extremely important. I would change the subtitle to “Know yourself”, reframing it to accepting that you / I need to know my self well (my strengths, weaknesses, my mental models, my skills, relationships, resources, experience, etc.) to be able to define for me what it means “to build the right thing” to find/define agency.
  - I my self-picture is skewed, the defined thing to build doesn’t rest on the center of gravity of my grounding, which can topple the whole pyramid. An imagined self-picture is a weak foundation for AI Engineering.
  - This is probably always relevant, but with AI, the leverage is much higher and everything in-between is accelerated, so a mis-aligned grounding is worse than in normal situations.
  - Grounding can and should be recalibrated from time to time, as I / we grow with AI (and without), and this can probably be faster than without in certain areas. (I think it can foster faster learning)
  - for myself: I think I formerly did not build on thinks I know well, but changed with the recent project, but I still think I have not really a well-aligned grounding in many areas. eg. I think I try to tackle projects that are maybe too large for me. even then, self awareness can help re-calibrate my grounding by leaning the right things and get the right help from others.
- the capability level was a quick self-assessment what I know well and not well enough for the projects I am doing and found some gaps I should fill.
- on the trust level, I realized that I use mostly tests, and try to use observability, but certainly do review not deeply enough at the moment (cannot cope with the amount of code generated), and evals is something I learned about last year with Hamel’s

and Shreya's courses, but need to adapt it more (with coding itself, I want to go to a kind of regular maintenance mode with ai assisted coding and hope I can introduce evals, then, as the tasks get more repetitive and stable.
- on the leverage level, I started with the obvious things, like CI/CD, review agents in actions, but got back to a more hands-on linear approach. again, I started to build a kind of test harness and deploy and overview / ops harnesses are next.
- what's not on the picture, but what really worries me is that I struggle to finish work and actually launch. My launch fears are heightened by the high speed, leverage, and necessarily incomplete nature of trust in SWE and AIE especially. I don't have a good idea how to fix it (as I also haven't fixed it, yet for myself). Maybe, this is again on the Agency level: Knowing what to build, and - in addition - knowing what not to build and knowing when it's good enough (for now).

This makes the pyramid much more interesting, because it stops being primarily a skills hierarchy and becomes a model of **alignment under leverage**.

The strongest new idea is this:

> **AI does not remove the need for grounding. It magnifies the consequences of weak grounding.**

That gives the whole pyramid a clearer internal logic.

I would revise it roughly like this:

1. Grounding — Know yourself Not just "understand the tools," but understand your own center of gravity: strengths, weaknesses, mental models, skills, relationships, resources, experience, interests, constraints, and blind spots.

The key insight is that **Agency at the top has to be vertically aligned with Grounding at the bottom**.

If the thing you decide to build rests on an imagined version of yourself rather than the real one, the pyramid becomes structurally unstable.

That does not mean "only build what you already know." It means knowing the distance between where you are and what the work requires. That lets you deliberately:

- learn the missing skill,
- reduce scope,
- change the approach,
- bring in another person,
- use AI to close a tractable gap,
- or decide the project is currently too large.

And grounding is not static. It should be **recalibrated** as experience accumulates. AI potentially accelerates that loop because it lets you attempt more, encounter your limits sooner, and learn faster—provided you actually observe what those attempts reveal about you.

---

2. Capability — Be able to build with and without the magic

I like this change a lot.

"Without the magic" protects against cargo-cult AI engineering. "With the magic" acknowledges that working effectively with AI is itself now a real engineering capability.

This layer can almost become a periodic inventory:

> What can I actually do? What can I do with AI assistance? What do I merely appear able to do because AI is carrying the hidden complexity? Which gaps matter for the things I am trying to build?

Your own quick self-assessment seems like exactly the intended use of this layer.

---

3. Trust — Know why you believe it works

I might sharpen "Verify, evaluate & build trust" into something more epistemic:

Trust — Know why you believe it works

Your observation about code volume is important here. AI creates a very specific problem:

> **Generation scales faster than human review capacity.**

So "I review the generated code" cannot be the long-term answer.

Tests, observability, evals, static analysis, invariants, architecture constraints, review agents, canarying, and other mechanisms are partly ways of **turning trust from an artisanal activity into infrastructure**.

Your current progression actually sounds quite coherent:

tests → observability → deeper evals → increasingly repeatable maintenance/evaluation loops.

And the maintenance-mode idea is interesting: once parts of the system become repetitive and stable enough, they become much better candidates for systematic evals because expected behaviour becomes clearer.

---

4. Leverage — Build the system that builds the system

I think your experience here reveals another useful correction.

Leverage does not necessarily mean "maximum autonomy."

You tried CI/CD, review agents, Actions and more agentic machinery, then moved back toward a more hands-on linear workflow. That is not necessarily moving backward.

It may be **calibrated leverage**.

The goal is not:

> How autonomous can I make this?

but:

> Where does automation increase my effective control, and where does it reduce it?

Your test harness, deployment harness and ops/overview harnesses feel like a very mature direction because they increase the leverage of both **you and the AI** while simultaneously strengthening Trust.

That suggests an important relationship between the layers: the pyramid isn't merely sequential. **Higher layers should reinforce lower ones.**

And then there is the really interesting missing issue:

5. Agency — Know what to build, what not to build, and when to stop

I think this belongs squarely at the top.

Perhaps the original wording "Build the right thing" is actually too optimistic, because in real work you almost never know with certainty what the right thing is.

Something like this may be stronger:

Agency — Choose what deserves to exist

with three questions underneath:

What should I build? What should I not build? When is it good enough to release?

That makes **finishing and launching** part of Agency rather than treating execution anxiety as an unrelated personal problem.

And I think your intuition about AI making this harder is correct in an important conceptual sense.

AI increases:

- the number of things you *could* build,
- the speed at which scope can expand,
- the amount of code you can create,
- the surface area you need to trust,
- and the number of improvements you can imagine before launch.

So the stopping problem gets worse.

There is always another prompt.

Another refactor.

Another agent.

Another edge case.

Another feature.

Another test.

Another architectural improvement.

At some point, **"good enough for now" becomes an engineering capability**.

I might even make that explicit in the model:

> **Agency is the ability to impose finite boundaries on effectively infinite possibility.**

That may be one of the defining skills of AI engineering.

This also gives you a powerful overall thesis for the pyramid:

> **AI Engineering is not primarily about maximizing AI leverage. It is about keeping leverage aligned with reality.**
>
> Grounding tells me who I am and what I have to work with. Capability tells me what I can build. Trust tells me what I can reasonably believe. Leverage determines how far and fast I can move. Agency determines where I choose to go—and when I choose to stop.

And then there is one relationship I would probably draw visually into the next version:

a vertical "alignment / center of gravity" line through the entire pyramid.

If Agency is far off-center from Grounding, increasing Capability, Trust infrastructure and Leverage might actually make the problem worse: **you become extremely effective at**

moving quickly toward something that is poorly aligned with who you are, your resources, or the problem you actually want to solve.

Conversely, the ideal isn't a permanently fixed pyramid. It is something more like a **self-correcting structure**:

Act → learn → update grounding → reassess capability → improve trust → recalibrate leverage → reconsider agency → act again.

That loop feels like the deeper model you are arriving at.

And I think your launch problem deserves visible space in the next PDF rather than being buried in the notes. I would add under Agency:

Choose • Bound • Finish • Launch

with perhaps this handwritten prompt alongside it:

"What does good enough for now mean?"